

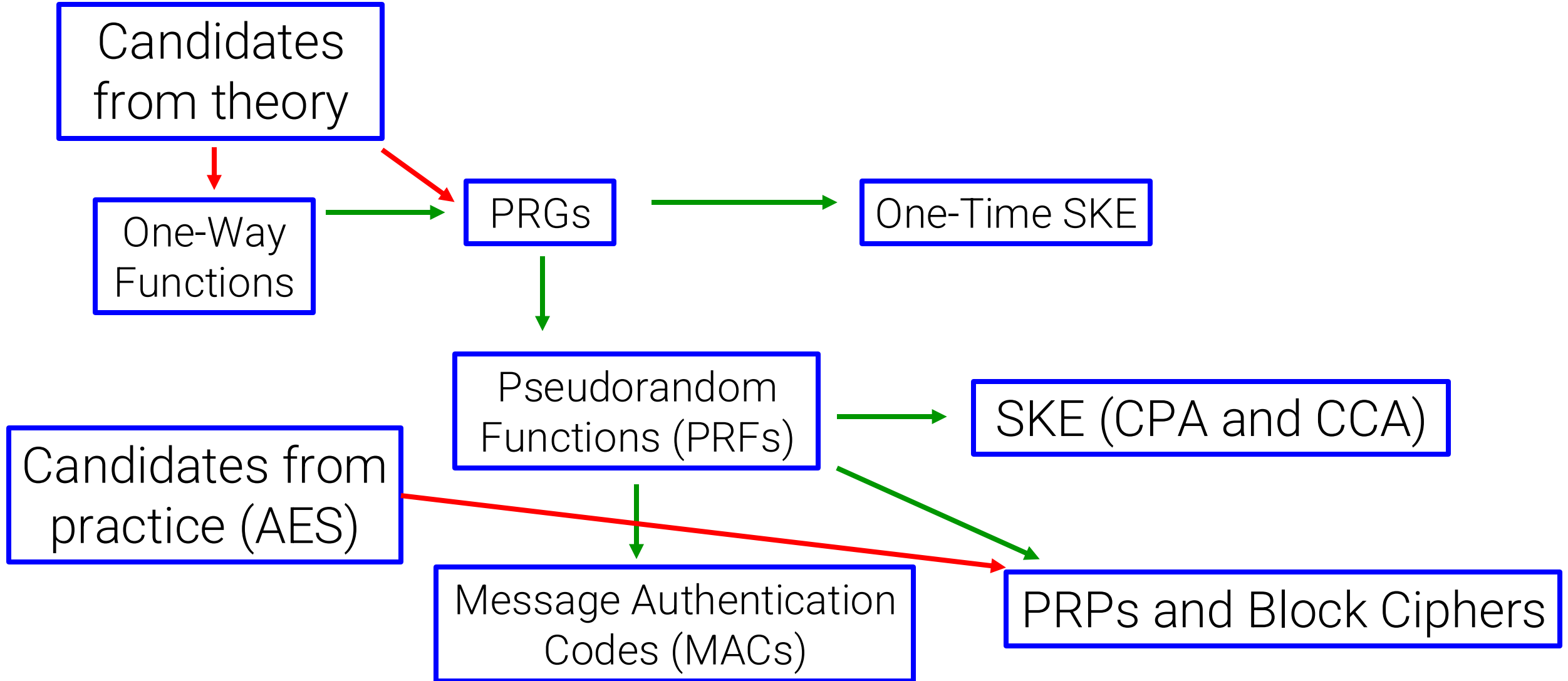
COS 433: Cryptography

Announcements:

- Problem Set 2 due tomorrow
- Problem Set 3 due date will be delayed due to fall break
- Lecture notes on computational number theory (by Dana Angluin) on the course website

Recap

Computationally Secure Cryptography



Next: Number-Theoretic Cryptography

Linear algebra over $\mathbb{Z}_2$

Security: dimension ($\mathbb{Z}_2^\lambda$)

Before

Arithmetic over $\mathbb{Z}_N$

Security: $N \approx 2^\lambda$

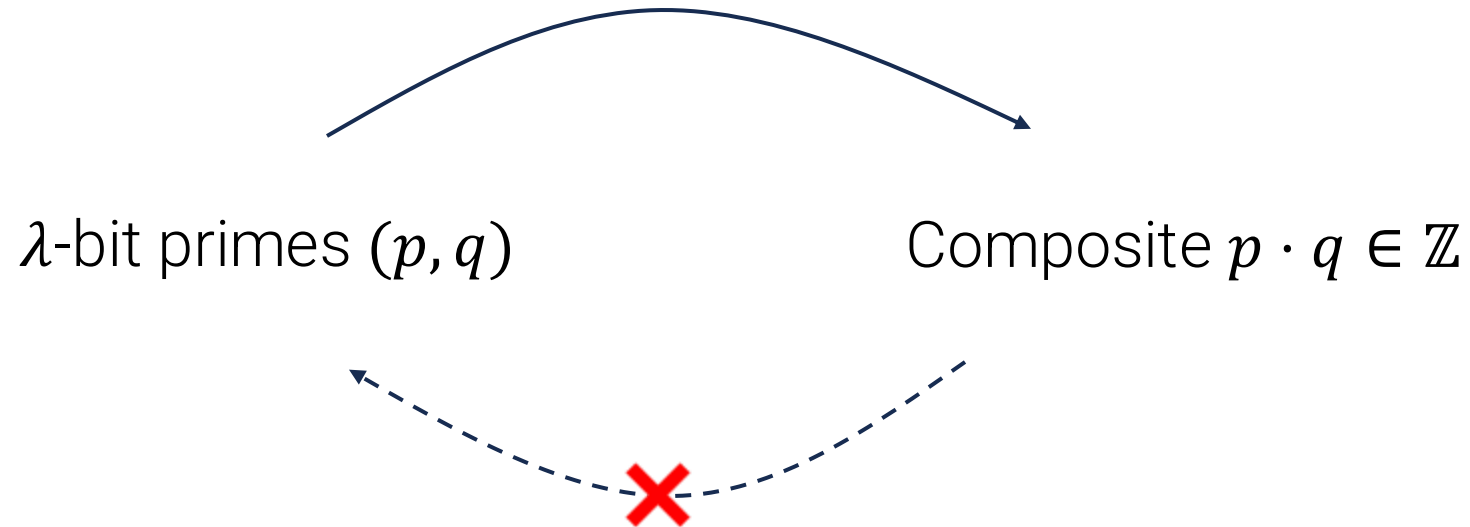
Now

Why Number-Theoretic Cryptography?

- Algorithmic number theory is over 2000 years old → good source of **cryptographic candidates**
- **Direct constructions** of more powerful primitives (SKE and PRFs)
- Used to build **public-key cryptography** (even **in practice**)

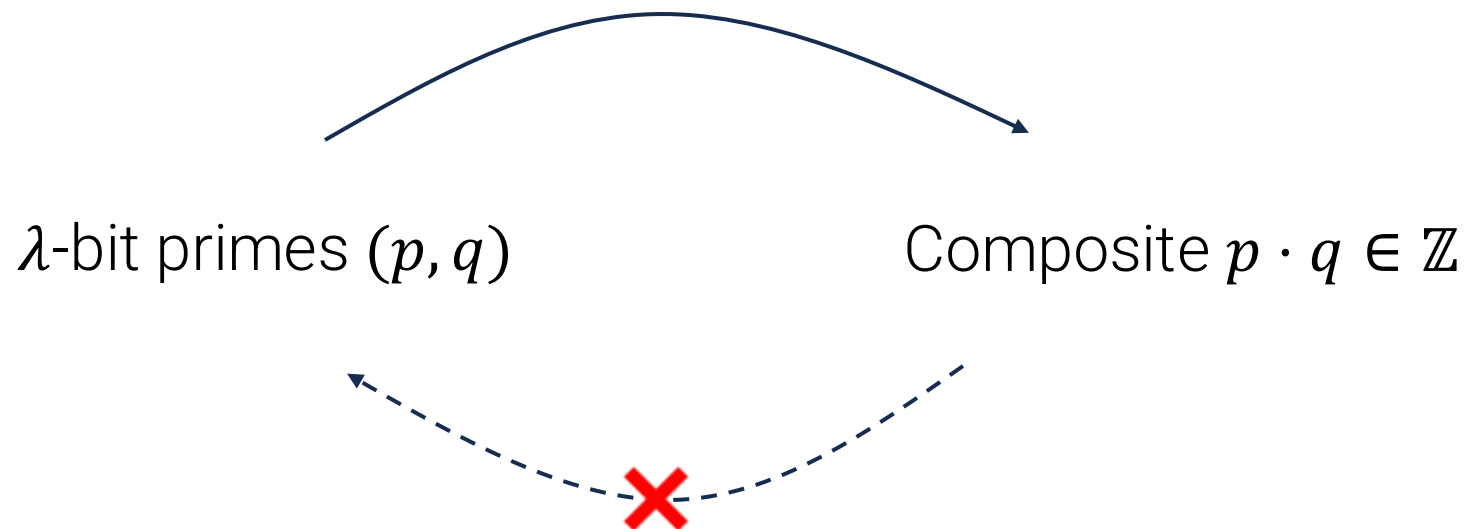
OWFs based on Factoring

OWFs based on Factoring



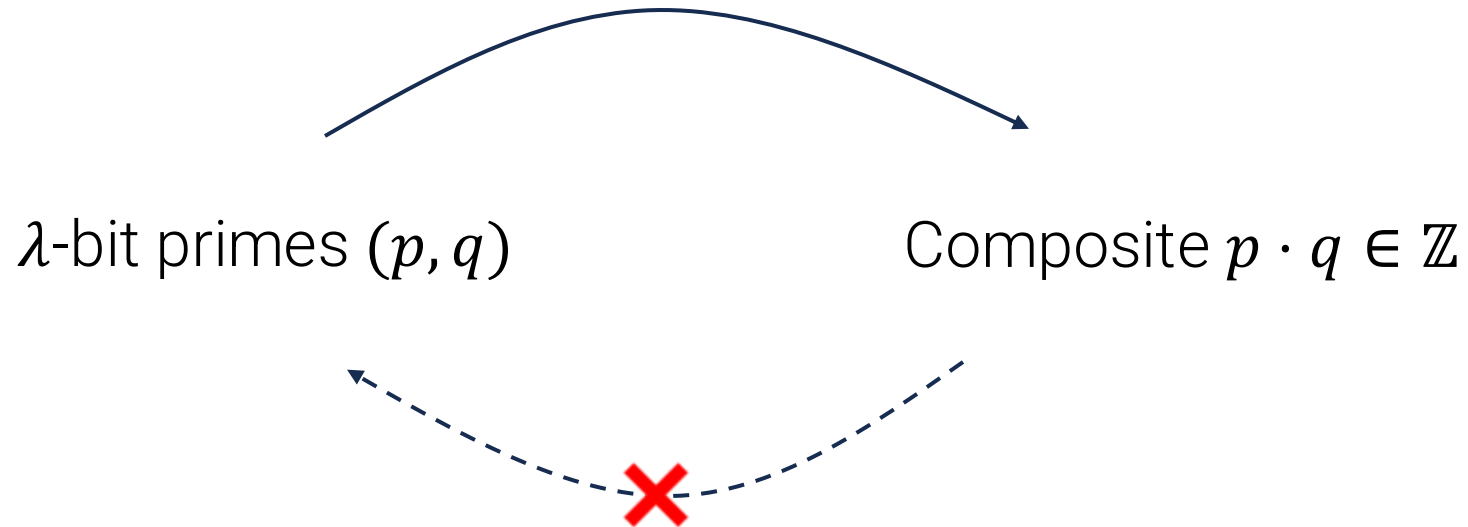
- Integer multiplication is **easy** ($O(\lambda^2)$)
- Prime factorization is believed to be **hard** (p, q random λ -bit primes)

OWFs based on Factoring



- How do we turn this into an actual OWF? (Need binary string domain)
- **Fact 1:** roughly $1/\lambda$ fraction of λ -bit integers are primes (Prime Number Theorem)
- **Fact 2:** primality testing is in P (Miller-Rabin, Agrawal-Kayal-Saxena)

OWFs based on Factoring



$f: \{0,1\}^{2 \cdot \lambda^3} \rightarrow \{0,1\}^{2 \cdot \lambda}$:

- Interpret input as $(p_1, p_2, \dots, p_{\lambda^2}, q_1, q_2, \dots, q_{\lambda^2})$
- For i from 1 to λ , check if p_i, q_i are prime.
 - Set p to be the first prime p_i , same for q
- Output $p \cdot q$

Key point: (p, q) extremely close to a uniformly random pair of primes

Plan for today

- Understand the hardness of factoring (a little) better
 - Crash course in computational number theory
- Find better number theoretic cryptographic constructions

Algorithms for Factoring N (history)

1. Trial division

- For a from 2 to $N - 1$, check if a divides N evenly
- Running time: $N \cdot \text{poly}(\lambda) = 2^\lambda \cdot \text{poly}(\lambda)$ for $N \leq 2^\lambda$
- Optimization: only check a from 2 to $\sqrt{N}$. Running time: $2^{\lambda/2} \cdot \text{poly}(\lambda)$

2. Sieve of Eratosthenes

- Only divide N by **primes**
- Build a table of all primes up to $\sqrt{N}$
- Bonus: **table is reusable**
- More practical but asymptotically similar

1	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

Algorithms for Factoring N

3. Fermat's Method

- Find integers a, b such that $N = a^2 - b^2 = (a + b)(a - b)$
- Requires brute force up to $a, b = \sqrt{N}$ (slow)

4. Congruence of squares (best known approach)

- Find a, b such that $a^2 - b^2 \equiv 0 \pmod{N}$ but $a \not\equiv \pm b \pmod{N}$
- Then, $\gcd(a + b, N)$ is a proper divisor of N
 - Euclid's algorithm computes $\gcd$ in $\text{poly}(\lambda)$ time
- How to find (a, b) ? Complicated
 - Quadratic Sieve [Pomerance '81] runs in time $2^{O(\sqrt{\lambda \cdot \log \lambda})}$
 - Number Field Sieve [Pollard '88] runs **heuristically** in time $2^{O(\sqrt[3]{\lambda \cdot \log^2(\lambda)})}$

Algorithms for Factoring N

Heuristic best known time complexity of factoring: $2^{O(\sqrt[3]{\lambda \cdot \log^2(\lambda)})}$

- Current factoring challenge: random $p, q \leq 2^\lambda$ with $\lambda = 512$
- “Safe” choices: $\lambda = 1024$ (until 2030), $\lambda = 2048$ (after 2030)

Hardness of Factoring

Evidence for factoring being computationally hard?

- Mathematicians have been trying to factor for 2000+ years
- The world has been trying to factor big numbers since at least 1978
- Relate factoring to **other problems** in computational number theory

Theorem:

Factoring $N \iff$ Solving **quadratic equations** over $\mathbb{Z}_N$

Arithmetic over $\mathbb{Z}_N$

- $\mathbb{Z}_N$ is a **commutative ring**
 - Addition and multiplication operations, commutative and associative
 - Additive identity 0, additive inverses $a + (-a) = 0$
 - Multiplication distributes over addition
- $\mathbb{Z}_N$ is an **effective** family of rings (we can compute on $\mathbb{Z}_N$)
 - Elements have a canonical string representative
 - Addition, subtraction and multiplication take $\text{poly log}(N) = \text{poly}(\lambda)$ time.

Arithmetic over $\mathbb{Z}_N$

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

- Case 1: linear equations

Given a, b , find x such that $a \cdot x \equiv b \pmod{N}$

- Solution exists iff $\gcd(a, N)$ divides b
- If $\gcd(a, N) = 1$ (wlog), there exists a^{-1} such that $a \cdot a^{-1} \equiv 1 \pmod{N}$
 - Extended Euclidean algorithm finds a^{-1} in $\text{poly}(\lambda)$ time
 - $x = a^{-1} \cdot b$
- For later: define $\mathbb{Z}_N^\times = \{a \in \mathbb{Z}_N : \gcd(a, N) = 1\}$ (multiplicative group)

Arithmetic over $\mathbb{Z}_N$

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

- Case 2: quadratic equations

Given a, b, c , find x such that $ax^2 + bx + c \equiv 0 \pmod{N}$

Square root finding over $\mathbb{Z}_N$

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

- Case 2: quadratic equations (simplifies to:)

Given a , find x such that $x^2 \equiv a \pmod{N}$

Claim 1: **easy** if N is prime

Claim 2: **easy** if factoring N is **easy**

Claim 3: **hard** if factoring N is **hard**

Square root finding over $\mathbb{Z}_N$

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

- Case 2: quadratic equations (simplifies to:)

Given a , find x such that $x^2 \equiv a \pmod{N}$

Claim 1: **easy** if N is prime

Fermat's little theorem: $a^p \equiv a \pmod{p}$ for all a . $a^{p-1} \equiv 1$ for all nonzero a .

Proof: $(x + y)^p = \sum_{k=0}^p \binom{p}{k} x^k y^{p-k} \equiv x^p + y^p \pmod{p}$ by the binomial theorem.

Square root finding over $\mathbb{Z}_N$

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

- Case 2: quadratic equations (simplifies to:)

Given a , find x such that $x^2 \equiv a \pmod{N}$

Claim 1: **easy** if N is prime

Special case: if $N = p \equiv 3 \pmod{4}$, then

$$\left(a^{\frac{p+1}{4}}\right)^2 \equiv a^{\frac{p+1}{2}} \equiv a \cdot a^{\frac{p-1}{2}} \pmod{p}$$

Fact: a is a square mod p if and only if $a^{\frac{p-1}{2}} \equiv 1 \pmod{p}$

Square root finding over $\mathbb{Z}_N$

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

- Case 2: quadratic equations (simplifies to:)

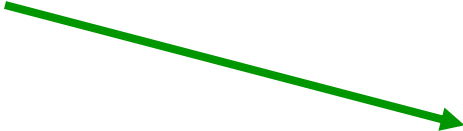
Given a , find x such that $x^2 \equiv a \pmod{N}$

Claim 1: **easy** if N is prime

Special case: if $N = p \equiv 3 \pmod{4}$ and a is a square mod p , then

$$\left(a^{\frac{p+1}{4}}\right)^2 \equiv a \pmod{p}$$

General case: exercise



Exponentiation is easy
by repeated squaring

Square root finding over $\mathbb{Z}_N$

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

- Case 2: quadratic equations (simplifies to:)

Given a , find x such that $x^2 \equiv a \pmod{N}$

Claim 2: **easy** if factoring N is **easy**

Proof: Chinese Remainder Theorem

If $\gcd(N, M) = 1$, then the map

$$x \mapsto (x \bmod N, x \bmod M)$$

Is an **efficiently invertible** bijection from $\mathbb{Z}_{NM} \rightarrow \mathbb{Z}_N \times \mathbb{Z}_M$

Square root finding over $\mathbb{Z}_N$

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

- Case 2: quadratic equations (simplifies to:)

Given a , find x such that $x^2 \equiv a \pmod{N}$

Claim 2: **easy** if factoring N is **easy**

Proof: special case $N = pq$

Solution exists iff a is a square mod p **and** mod q

Find solutions mod p and mod q (up to 2 each)

Piece together **(up to 4)** solutions mod N by inverting the CRT map

Square root finding over $\mathbb{Z}_N$

How hard is it to solve equations **in one variable** over $\mathbb{Z}_N$?

- Case 2: quadratic equations (simplifies to:)

Given a , find x such that $x^2 \equiv a \pmod{N}$

Key point: solutions (for $N = pq$) have the form

$$x \equiv \pm r \pmod{p}, x \equiv \pm s \pmod{q}$$

Square root finding over $\mathbb{Z}_N$

Given a , find x such that $x^2 \equiv a \pmod{N}$

Claim 3: **hard** (even on average) if factoring N is **hard**

Proof: by reduction. Suppose that A is an efficient algorithm with

$$\Pr_{\substack{p,q \\ x \leftarrow \mathbb{Z}_N^\times}} [A(pq, x^2) = y \text{ such that } y^2 \equiv x^2 \pmod{N}] = \epsilon(\lambda)$$

Goal: build A' that factors N with non-negligible probability

Define:

$$A'(N) = \gcd(N, x + A(N, x^2))$$

Square root finding over $\mathbb{Z}_N$

Goal: build A' that factors N with non-negligible probability

Define:

$$A'(N) = \gcd(N, \textcolor{red}{x} + A(N, \textcolor{red}{x}^2))$$

Analysis: let $\textcolor{green}{y} = A(N, \textcolor{red}{x}^2)$.

- 1) We know that $\textcolor{green}{y}^2 \equiv \textcolor{red}{x}^2$ with probability ϵ .
- 2) There are four possible solutions for a fixed $\textcolor{green}{z} = \textcolor{red}{x}^2$
- 3) Subroutine A does not know, given $\textcolor{green}{z}$, which of the four $\textcolor{red}{x}$ that A' used.

Square root finding over $\mathbb{Z}_N$

Goal: build A' that factors N with non-negligible probability

Define:

$$A'(N) = \gcd(N, x + A(N, x^2))$$

Analysis: let $y = A(N, x^2)$.

1) We know that $y^2 \equiv x^2$ with probability ϵ .

$$\Pr[y \equiv x \pmod{N} \mid y^2 \equiv x^2] = 1/4$$

$$\Pr[y \equiv -x \pmod{N} \mid y^2 \equiv x^2] = 1/4$$

Square root finding over $\mathbb{Z}_N$

Goal: build A' that factors N with non-negligible probability

Define:

$$A'(N) = \gcd(N, x + A(N, x^2))$$

Analysis: let $y = A(N, x^2)$.

1) We know that $y^2 \equiv x^2$ with probability ϵ .

$$\Pr[y \notin \{\pm x \pmod{N}\} \mid y^2 \equiv x^2] = 1/2$$

2) But for any solution y , we have $y \equiv \pm x \pmod{p}$ and $\pmod{q}$

3) For these y , $y + x$ is divisible by exactly one of p and q !

Theorem:

Factoring $N \iff$ Solving **quadratic equations** over $\mathbb{Z}_N$

Back to constructing cryptography

OWFs based on Factoring

We believe this (modulo domain issues) is a OWF:

$$f(p, q) = p \cdot q$$

OWFs based on Factoring

Here's another candidate OWF:

$$f(p, q, x) = (pq, x^2 \bmod pq)$$

More secure: can't even find a square root x , let alone (p, q)

Question: is this a "PRG?"

$$f(\textcolor{red}{p}, \textcolor{red}{q}, \textcolor{red}{x}) \approx_c (\textcolor{red}{pq}, \textcolor{red}{u} \bmod \textcolor{red}{pq})?$$

Continue next time